Constella: A Novel Framework for Cost-Efficient Distributed AI Inference in LEO Space Data Centers

Andrija Stanisic[1[0009−0007−2120−1142]], Milos Gravara[1[0009−0006−7986−5243]], Juan Luis Herrera[1[0000−0002−2280−2878]], and Stefan Nastic[1[0000−0003−0410−6315]]

Distributed Systems Group, Technische Universität Wien, Austria
{a.stanisic,m.gravara,j.gonzalez,s.nastic}@dsg.tuwien.ac.at

Abstract. Space data centers built from Low-Earth Orbit satellite constellations are gaining increasing attention as a scalable computing infrastructure. With access to abundant solar energy and high-throughput optical inter-satellite links, such constellations can deliver computational services to remote and rural regions that lack terrestrial network infrastructure. However, managing satellite constellations that combine heterogeneous satellite roles introduces a cost optimization challenge. Determining the appropriate constellation size and satellite role ratio for a given workload is challenging, as over-provisioning processing satellites increases system cost, while under-provisioning limits system efficiency. To enable cost-efficient execution of AI inference workloads in such space data centers, we present Constella, a novel framework for distributed AI inference in LEO satellite constellations. Constella comprises an offline resource identifier that determines the optimal ratio of processor-to-communicator satellites and an online assignment algorithm. The algorithm utilizes constellation telemetry to adaptively route data within the constellation and to ground stations. We evaluate Constella on a real-world satellite dataset across scenarios of increasing complexity. Results demonstrate a reduction in system cost by up to two orders of magnitude and lower end-to-end inference latency by up to 2.7× compared to other approaches, while maintaining no less than 81.9% inference success rate.

Keywords: LEO space data centers · Distributed inference · Edge-Cloud-Space continuum · DNN model partitioning.

1 Introduction

Low-Earth Orbit (LEO) satellite constellations are emerging as a viable platform for building space data centers [3]. Equipped with solar arrays, optical inter-satellite links (ISLs), and compute accelerators, these constellations benefit from solar energy yields up to eight times higher than on Earth, while being capable of providing significant compute power [3]. As artificial intelligence (AI) is widely regarded as a foundational general-purpose technology, the demand for computing infrastructure supporting AI workloads is projected to grow by orders of magnitude in the coming decades [3]. This demand arguably exceeds what terrestrial energy sources alone can sustain, making space data centers a potential solution for provisioning AI compute at scale [3]. Furthermore, LEO constellations are already pivotal for Earth Observation (EO) applications such as wildfire detection and disaster management [18]. Traditional EO architectures follow a "bent pipe" design, where each satellite captures imagery and transmits

it to the ground station during intermittent communication windows [6]. To reduce the volume of transmitted data, recent approaches opt for splitting the AI model between the satellite and the ground station [6]. Here, the split layer dictates how much computation is performed on-board and how much intermediate data has to be transmitted to the ground station for tail-end inference [6].

However, traditional architectures concentrate both computational and communication loads on a single resource-constrained satellite. This is why existing constellations, such as SpaceDataHighway of the European Data Relay System (EDRS) [2], introduce distinct satellite roles to distribute heavy workloads and energy-expensive downlink transmission among different satellites. Satellites equipped with processing hardware are more expensive to deploy and operate than their transmission-focused counterparts [15, 7]. This cost asymmetry makes the ratio of processor to communicator satellites a key design decision. Since space data center infrastructure remains substantially more expensive than terrestrial alternatives [3], cost reduction is expected to contribute to wider adoption. For instance, over-provisioning processors inflates deployment cost, while under-provisioning communicators creates transmission bottlenecks. The problem is further complicated by the intermittent nature of LEO downlink. Due to high orbital velocities, direct ground contact is restricted to brief communication windows [30], and processed results must otherwise be buffered on-board [5]. Modern LEO constellations mitigate this limitation through high-throughput ISLs that reach up to 100 Gbps [17, 4] while consuming significantly less energy than downlink transmission [22]. This energy asymmetry enables a cost-efficient alternative where intermediate results are forwarded via ISL to satellites with imminent ground access, rather than requiring each satellite to independently perform energy-expensive downlink [25]. Realizing this cost advantage requires addressing two research challenges, namely determining the minimum-cost subset of processor and communicator satellites from a given constellation that satisfies inference workload constraints, and coordinating their ISL communication and ground station transmission during execution.

In this work, we propose Constella, a two-phase framework for cost-efficient distributed inference across heterogeneous LEO satellite constellations. First, in an offline phase, a constellation resource identifier jointly determines the Deep Neural Network (DNN) split layer and the minimum-cost constellation required to carry out the defined inference workload. This phase includes finding the cost-optimal number of processor and communicator satellites that must be used from a space data center or deployed. Then, in an online phase, a latency-aware assignment algorithm dynamically assigns each processor to a communicator upon every inference task, leveraging ISL communication to minimize end-to-end delivery latency. Specifically, the main contributions of this work include:

- **Constella**: A novel two-phase framework for planning and executing cost-efficient distributed inference across heterogeneous LEO constellations. Constella separates satellites into processors and communicators, jointly optimizing constellation composition offline and processor-to-communicator assignment online, upon workload deployment. Evaluation on a real-world satellite dataset shows that Constella maintains at least 81.9% inference success while

reducing deployment cost by one to two orders of magnitude, mean end-to-end latency by up to 2.7×, and total energy consumption by up to 74× across scenarios of varying complexity, compared to traditional baselines.

- **OCRI**: An **O**ffline **C**onstellation **R**esource **I**dentifier that determines the DNN split layer and the cost-minimal constellation configuration. OCRI, aimed at the offline phase of Constella, formulates a constrained optimization problem, which is then linearized into a mixed integer linear program and solved for global optimality. Across different scenarios, OCRI exhibits low execution overhead, with mean runtimes below 24 ms per configuration.
- **LIA**: A **L**atency-aware **I**SL **A**ssignment algorithm for the online phase of Constella that minimizes end-to-end inference latency by dynamically assigning processors to communicators. LIA leverages gossip-based state dissemination to construct a set of eligible communicators based on buffer capacity, energy feasibility, and communication window availability, then selects the satellite with the earliest ground contact. Across scenarios, LIA remains lightweight, with cumulative execution time for all assignment decisions within one orbit below 5 ms.

The remainder of this paper is organized as follows. Section 2 motivates the work and positions it within the existing literature. Section 3 details considered system model and the Constella framework, focusing on its key underlying mechanisms. Experimental results are presented in Section 4, while Section 5 concludes the work with future research directions.

2 Motivation & Related Work

This section motivates the problem through an EO use case in Section 2.1. Section 2.2 analyzes the state-of-the-art and positions our work within this context.

2.1 Motivational Use Case

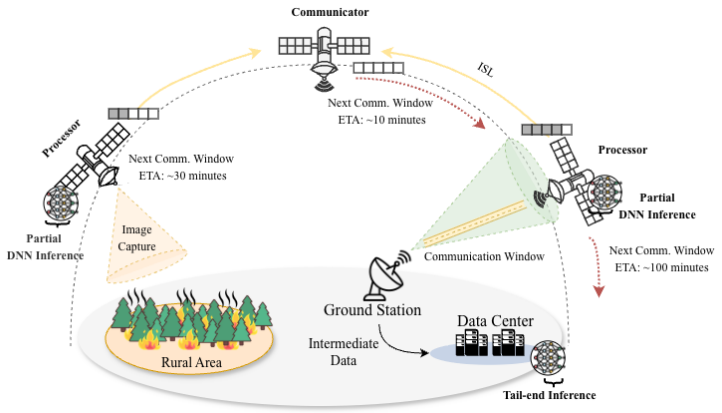

Fig. 1: Motivational scenario for disaster detection.

To better illustrate the motivation behind Constella, we present a use case on natural disaster prediction shown in Figure 1. This motivational use case

serves as a showcase of the problems tackled by Constella in space data centers. To ensure the realism of the use case, the domain information is based on the proposal of Adeel *et al.* [1], and data for the technical and satellite aspects were extracted from real satellite constellations and DNN architectures [10, 20].

Natural disasters such as wildfires, earthquakes, and flooding cause significant human and economic losses worldwide [8]. These events result in significant annual fatalities and economic losses that frequently exceed hundreds of billions of USD [8]. The primary objective in mitigating these impacts is early detection and prediction to ensure timely emergency responses [31]. Traditional monitoring approaches address this by deploying ground-based sensor networks that continuously collect environmental data and propagate it into a terrestrial computing continuum for processing, analysis, and dissemination [1]. However, these approaches rely on the availability of reliable network connectivity. Many rural and remote regions lack such infrastructure, often due to the prohibitive cost of deploying and maintaining network equipment in areas with limited economic return [9]. At the same time, rural regions are disproportionately affected by natural disasters [23]. This creates a fundamental gap where monitoring and processing infrastructures are least available where they are needed the most. LEO satellite constellations offer an infrastructure-independent alternative. Equipped with imaging sensors and compute hardware, LEO satellites can capture imagery of affected regions, execute on-board inference, and transmit results to ground stations without relying on terrestrial networks [3, 6]. This way, the satellites can achieve the same role as terrestrial approaches, providing continuous Earth observation and disaster management over areas that lack sufficient infrastructure.

For this motivational use case, consider a scenario in which governments of rural regions develop a DNN model for early wildfire detection and deploy it using the services of a LEO space data center. The data center offers two types of satellites at different price points, specifically processors and communicators. The processors capture Earth imagery, execute on-board inference, and perform downlink transmission, while the communicators are dedicated to downlink transmission at a lower cost. Both satellite types are equipped with high-throughput ISL terminals, enabling intra-orbit data propagation. Given these two satellite types, the first step is determining how many satellites of each type are required to serve the application. However, determining the right constellation dimensioning for a given workload is not trivial. With N satellites fulfilling one of the two previously defined roles and a DNN architecture comprising $|L|$ candidate split layers, the number of possible configurations is $2^N \cdot |L|$. For example, given the IRIDIUM [10] constellation, which contains $N = 66$ satellites and a DNN architecture such as ResNet50 with $|L| = 182$ layers, the possible configurations reach approximately $2^{66} \cdot 182 \approx 1.35 \cdot 10^{22}$. For this relatively small constellation and simple neural network architecture, the search space is already intractable for manual exploration. Modern LEO constellations often comprise thousands of satellites [4], and state-of-the-art DNNs are becoming more complex [21], which further amplifies the combinatorial growth of the problem. Hence, an automated solution for cost-efficient constellation composition becomes necessary.

Despite optimal constellation composition, runtime operation remains challenging. Each processor must decide which communicator to forward intermediate data to. The goal is minimizing end-to-end latency, as it can be crucial for timely disaster response [31]. Figure 1 illustrates varying communication window timings among communicators. Some enable immediate downlink, while others require on-board buffering. Targeting the nearest communicator minimizes transfer delay but risks long buffering. Conversely, targeting the earliest ground contact risks overload. Processors can execute direct downlink transmission, bypassing communicators, but this process requires sufficient energy for both inference and transmission. This combination of objectives and dynamic conditions makes the assignment problem unsuitable for a static optimization.

2.2 Related Work

Space data centers built from LEO satellite constellations have recently emerged as a recognized computing paradigm. Beals et al. [3] present a vision for a future space AI infrastructure, projecting that terrestrial energy sources alone will be insufficient to sustain growing AI demand at scale. The problem of executing DNN inference across satellite-ground architectures has received considerable attention. Chen et al. [6] propose SLICE, which jointly optimizes the DNN split layer and transmission scheduling to minimize energy consumption within a fixed satellite deployment. Yao et al. [28] present LEOEdge, an inference platform that generates per-satellite models and applies layered scheduling to distribute tasks across a constellation. Xu et al. [27] propose COIN-LEO, which uses deep reinforcement learning to split a DNN into sequential submodels deployed across multiple homogeneous satellites for collaborative inference. In all of these works, the satellite constellation is treated as a fixed input. The number of satellites and their roles are assumed rather than determined. Where the DNN split layer is optimized, it is done within a given constellation rather than jointly with it. As shown in Section 2.1, the joint configuration space grows combinatorially with constellation size, making manual selection intractable. To the best of our knowledge, no existing work addresses this joint optimization under energy and deployment cost constraints, which represents the gap addressed in this work. Similar resource configuration problems are well-established in other distributed systems domains, such as the controller placement problem in SDN [11], further motivating a principled solution for the satellite setting.

Beyond constellation configuration, routing data within LEO constellations via ISLs to reduce transmission latency has also attracted recent research efforts. Jia et al. [12] propose a collaborative scheme that offloads data among satellites via ISLs before ground contact, jointly scheduling offloading and downloading to maximize ground station throughput. Li et al. [14] address the last-hop scheduling problem in large-scale constellations, proposing a centralized algorithm that issues download commands to satellites within the communication window of a ground station. In both works, routing decisions are computed centrally over a pre-planned schedule and do not operate at the granularity of individual inference tasks. To the best of our knowledge, no existing work addresses decentralized, per-task assignment of inference results from processor to communicator

satellites under joint constraints on buffer capacity, energy feasibility, and communication window periods, which represents the gap addressed in this work.

3 Constella Framework Overview

This section first introduces the system model and its underlying assumptions in Section 3.1. It then presents the Constella framework and its two key components, OCRI and LIA, described in Section 3.2 and Section 3.3, respectively. OCRI executes on ground infrastructure prior to workload deployment and determines the minimum-cost constellation configuration that satisfies workload and environment constraints. The resulting configuration is then managed by LIA, which dynamically assigns processors to communicators during execution.

3.1 System Model

For this model, let a LEO satellite constellation comprise $N \in \mathbb{N}_1$ satellites and a dedicated ground station. Drawing from the architectural decoupling established in constellations like SpaceDataHighway [2], we propose partitioning the LEO constellation into two disjoint satellite roles. First, the processors, which are satellites equipped with imaging sensors, on-board compute hardware, ISL terminals, and antennas for downlink transmission. Second, the communicators, which carry ISL terminals and antennas that enable downlink transmission. Processors are more expensive to launch and use due to their complexity and specialized hardware [7]. Given this, we assume that, in the case of a LEO data center, the price for using processors, denoted as $\alpha \in \mathbb{R}^+$, is not lower than $\beta \in \mathbb{R}^+$, which denotes the price of usage of communicators. The LEO data center provides a set of processors, denoted by $\mathcal{X}$, and communicators, denoted by $\mathcal{Y}$, where $N = |\mathcal{X}| + |\mathcal{Y}|$. The numbers of activated processors and communicators are denoted by X and Y, respectively, where $X \leq |\mathcal{X}|$ and $Y \leq |\mathcal{Y}|$. Processing hardware efficiency is characterized by the energy cost of computation $p \in \mathbb{R}^+$ $(mW/FLOP)$ and the energy cost of transmission $q \in \mathbb{R}^+$ (mW/bit), both of which are treated as a priori known parameters. Additionally, energy budgets per orbit for processors and communicators, denoted as $E^{proc}_{budget}, E^{comm}_{budget}$ respectively, and downlink channel throughput, denoted as R^{max}, are also considered to be known a priori. These assumptions are based on the information availability of traditional cloud-providing platforms.

Regarding the constellation topology, we consider full pairwise connectivity via multi-hop ISL routing [24], with shortest-path routing treated as orthogonal to this work. We consider a static deployment snapshot where X processors and Y communicators are uniformly interleaved across the orbital plane. This inherently minimizes processor-to-communicator distances, while joint spatial role optimization remains promising future work, as acknowledged in Section 5. To enable decentralized assignment during execution, we assume a lightweight ISL-based gossip protocol. Because ISL transmissions incur negligible latency and energy costs compared to ground transmission [22], communicators can periodically broadcast their current state without depleting limited resources. Specifically, each activated communicator $y \in \mathcal{Y}$ disseminates its buffer occupancy $buff(t)_y$, remaining energy budget $E^{comm}_{budget}(t)_y$, and time to next ground contact

t_y^g. The gossip period is strictly shorter than Δ_t, ensuring all processors maintain a consistent view of communicator states before each assignment decision.

Furthermore, the orbit period $\mathcal{T}$ comprises computation $\mathcal{T}^{comp}$, communication $\mathcal{T}^{comm}$, and idle $\mathcal{T}^{idle}$ phases, where $\mathcal{T} = \mathcal{T}^{comp} + \mathcal{T}^{comm} + \mathcal{T}^{idle}$. The $\mathcal{T}^{comp}$ corresponds to the orbital segment traversing an area of interest (AOI), that is, regions requiring critical monitoring such as earthquake-impacted zones. For simplicity, we consider a single shared AOI across all processors. During $\mathcal{T}^{comp}$, processors capture imagery and execute DNN inference at fixed intervals Δ_t, yielding $I_{max} = \lfloor \mathcal{T}^{comp}/\Delta_t \rfloor$ potential tasks per orbit. We assume that Δ_t is sufficient for full DNN execution on-board, which excludes inference latency from optimization. However, this remains a potential future research direction, as acknowledged in Section 5. Conversely, during $\mathcal{T}^{comm}$, processors or communicators transmit accumulated results to the ground station. To avoid the complexities associated with boundary-case data arrivals, any satellite entering $\mathcal{T}^{comm}$ is considered unavailable to receive additional data for downlink process, though it remains available as an ISL relay node. We assume that there is no overlap between $\mathcal{T}^{comm}$ and $\mathcal{T}^{comp}$, as the ground station is not considered to be part of the AOI (e.g., because it has access to Internet infrastructure). During $\mathcal{T}^{idle}$, satellites traverse orbital segments outside both the AOI and ground station visibility, performing neither inference nor downlink transmission.

Finally, the DNN model architecture is represented as an ordered set of layers $L = \{1, \ldots, |L|\}$. The split layer $l \in L$ defines the boundary of on-board execution. All layers from input up to and including layer l are executed on the satellite. In case $l = |L|$, complete inference is executed on-board and only the final result is transmitted. The split layer induces two workload functions. The cumulative FLOPs, denoted with $W(l) \in \mathbb{R}^+$, needed to execute the first l layers, and $D(l) \in \mathbb{R}^+$, the intermediate output size at layer l in bits.

3.2 OCRI: Offline Constellation Resource Identifier

The primary objective of the OCRI is to identify a minimum-cost constellation composition that serves a given inference workload under energy constraints. The workload is defined by a DNN architecture and a target number of inference tasks per orbit I_{total}. The optimization goal is to identify a triple (l, X, Y) that minimizes deployment cost while complying with the energy constraints of each satellite. The resulting optimization problem is not linear due to the dependency of the workload functions $W(l)$ and $D(l)$ on the discrete split layer l. Since $W(l)$ and $D(l)$ are determined by the DNN architecture, they do not follow a closed-form expression over l and cannot be treated as continuous variables. To linearize the problem, we introduce binary variables $z_l \in \{0, 1\}$ for each candidate layer $l \in L$, with $\sum_{l \in L} z_l = 1$ ensuring that exactly one split layer is selected. The workload functions are then expressed as linear combinations $\sum_{l \in L} W(l) \cdot z_l$ and $\sum_{l \in L} D(l) \cdot z_l$, respectively, where $W(l)$ and $D(l)$ become known constants for each l. This yields the mixed-integer linear program (MILP) in Equation 1, jointly determining the split layer and the processor and communicator counts.

$$
\begin{aligned}
&\min_{X,Y,z} \quad \alpha \cdot X + \beta \cdot Y \\
&\text{s.t.} \\
&\quad \mathrm{c}_1: \quad (X+Y) \cdot R^{max} \cdot \mathcal{T}^{comm} \geq I_{total} \cdot \sum_{l \in L} D(l) \cdot z_l \\
&\quad \mathrm{c}_2: \quad X \cdot E^{proc}_{budget} \geq I_{total} \cdot \sum_{l \in L} W(l) \cdot z_l \cdot p \\
&\quad \mathrm{c}_3: \quad I_{total} \cdot (\sum_{l \in L} W(l) \cdot z_l \cdot p + \sum_{l \in L} D(l) \cdot z_l \cdot q) \leq X \cdot E^{proc}_{budget} + Y \cdot E^{comm}_{budget} \\
&\quad \mathrm{c}_4: \quad X \cdot I_{max} \geq I_{total} \\
&\quad \mathrm{c}_5: \quad X \in \mathbb{N}_1,\, Y \in \mathbb{N}_0,\, X \leq |\mathcal{X}|, Y \leq |\mathcal{Y}|, z_l \in \{0,1\}^{|L|} \ \wedge \sum_{l \in L} z_l = 1
\end{aligned}
\tag{1}
$$

The objective minimizes total deployment cost as a weighted sum of processor and communicator counts. Constraint c_1 ensures that the aggregate downlink capacity covers the total intermediate data generated per orbit. Constraint c_2 guarantees that processors have sufficient energy for their share of inference tasks. Constraint c_3 bounds the total energy consumption by requiring that the combined computation and communication energy does not exceed the aggregate energy budget of all satellites. Runtime load imbalance is handled by the online algorithm. Constraint c_4 bounds the per-processor task load by the maximum number of capture intervals within $\mathcal{T}^{comp}$. Constraint c_5 enforces integrality, requiring at least one processor while permitting constellations with no dedicated communicators, bounds processor and communicator counts to their respective pool sizes, and restricts the split point to L. Since OCRI executes offline on ground infrastructure with sufficient computational resources, solving the MILP to optimality is feasible, guaranteeing an optimal constellation configuration for the given workload.

3.3 LIA: Latency-Aware ISL Assignment

LIA operates during the online stage of Constella, receiving the optimal configuration (l, X, Y) from OCRI as input. The main objective of LIA is to assign processors to communicators in order to route each inference result to the ground station while minimizing end-to-end delivery latency and respecting energy constraints. LIA executes independently on each processor satellite at every capture interval Δ_t, making a per-task routing decision after on-board inference completes. Algorithm 1 formalizes this procedure.

At each interval, communicator states are available to all processors through the gossip protocol described in Section 3.1. Given the received buffer occupancy, remaining energy budget, and estimated time to the next ground contact for each communicator, LIA first constructs a set of eligible communicators $\mathcal{E}$ (line 1). A communicator is excluded if it has already entered its communication window and can no longer accept additional data for the current pass, if the accumulated buffer including the new data exceeds the downlink capacity $R^{max} \cdot \mathcal{T}^{comm}$, or if the communicator lacks sufficient energy to transmit the resulting buffered data (line 3). Communicators that do not violate any of these conditions are added to $\mathcal{E}$ (line 6). If $\mathcal{E}$ is non-empty (line 8), the processor selects the communicator with the minimum estimated time to ground contact (line 9) and then forwards

Algorithm 1 LIA

Require: $l, Y, R^{max}, q, E^{comm}_{buget}(t)_y, E^{proc}_{budget}(t), t^g_y, buff(t)_y, D(l), \mathcal{T}^{comm}$

1: $\mathcal{E} \leftarrow \emptyset$
2: **for all** $y \in Y$ **do**
3: **if** $t^g_y = 0 \ \vee \ buff(t)_y + D(l) > R^{max} \cdot \mathcal{T}^{comm} \ \vee \ (buff(t)_y + D(l)) \cdot q > E^{comm}_{buget}(t)_y$ **then**
4: **continue**
5: **end if**
6: $\mathcal{E} \leftarrow \mathcal{E} \cup \{c\}$
7: **end for**
8: **if** $\mathcal{E} \neq \emptyset$ **then**
9: $y^* \leftarrow \arg\min_{y \in \mathcal{E}} \ t^g_y$
10: Forward $D(l)$ to y^* via ISL
11: **else if** $E^{proc}_{budget}(t) \geq D(l) \cdot q$ **then**
12: Reserve energy $\rightarrow$ queue $D(l)$ for direct downlink
13: **else**
14: Transmission infeasible
15: **end if**

the result via ISL (line 10). If no communicator passes the eligibility checks, the processor falls back to direct downlink. However, there is a check if the processor has sufficient energy to transmit the data itself (line 11). In case it does, it reserves the required energy and stores the data for its own communication window (line 12). Otherwise, the transmission is infeasible (line 14). Since LIA operates without a central orchestrator, multiple processors may independently select the same communicator within the same interval. This can lead to transient buffer overloading, which can be resolved in subsequent intervals as updated states propagate through gossip protocol.

4 Evaluation

This section presents a series of experiments as means to evaluate Constella. Section 4.1 details the carried-out experiments, experimental frameworks, and evaluation objectives, while Section 4.2 present the results.

4.1 Experimental setup

To evaluate the performance of Constella, we compare it against two baseline approaches that represent prevalent design strategies. In the absence of existing approaches that address the same problem as Constella, we construct representative baselines that capture fundamental trade-offs between satellite resource planning and runtime communication assignment. Each baseline, like Constella, comprises an offline resource-identifying phase and an online assignment phase, enabling a fair comparison. The Naive Baseline (NB) utilizes all available satellites in the constellation ($X = |\mathcal{X}|, Y = |\mathcal{Y}|$) and selects the architectural midpoint of the DNN as the split layer ($l = \lfloor |L|/2 \rfloor + 1$), representing a heuristic that requires no optimization. During the online phase, each processor forwards its inference output to a fixed communicator determined by orbital position through a static, pre-computed assignment. The Traditional Baseline (TB) employs all available processing satellites ($X = |\mathcal{X}|, Y = 0$) and performs no on-board inference. Each satellite captures raw imagery and transmits it directly to the ground station during its communication window. This baseline effectively represents the traditional "bent pipe" architecture. Together, these two baselines

span the design space from over-provisioned split inference to a communication-only pipeline, enabling assessment of decisions made by Constella.

Table 1: Evaluation scenarios and constellation configuration.

Scenario Configuration					
Scenario	$\lvert\mathcal{X}\rvert$	$\lvert\mathcal{Y}\rvert$	I_{total}	Model	$\lvert L\rvert$
extra-small	8	2	100	alexnet	23
small	40	10	500	squeezenet1.0	67
medium	400	100	1000	resnet50	182
large	4000	1000	10000	swin-b	311
extra-large	8000	2000	100000	efficientnet-b0	329
Constellation Configuration					
Parameter(s)			Value(s)		Unit
$R^{\max}$			625 000		bps
p			$1.63 \cdot 10^{-13}$		Wh/FLOP
q			$9.89 \cdot 10^{-8}$		Wh/bit
α, β			$\alpha = 4.0,\ \beta = 1.0$		–
$E^{proc}_{budget}, E^{comm}_{budget}$			$E^{proc}_{budget} = 1.5, E^{comm}_{budget} = 5.0$		Wh
$\mathcal{T}^{comp}, \mathcal{T}^{comm}, \mathcal{T}^{idle}, \Delta_t$			$\mathcal{T}^{comp} = 5100, \mathcal{T}^{comm} = 300,\ \mathcal{T}^{idle} = 900,\ \Delta_t = 10$		s

The evaluation framework is implemented using the PyTorch ecosystem for model management. The OCRI optimization problem is formulated and solved using the `python-mip` [19] library. The `torchinfo` [29] library is used to extract per-layer metrics, specifically the cumulative floating-point operations $W(l)$ and the intermediate output data size $D(l)$ for each candidate split layer $l \in L$. All models are analyzed using a fixed input tensor shape $(3, 224, 224)$, representing the standard input for the considered model architectures [13]. To demonstrate the generality of the Constella, we evaluate it across five evaluation scenarios that vary in constellation size, inference workload, and DNN architecture complexity. Following established data center cost models [16], we adopt a α:β = 4:1 price ratio between processors and communicators, using the communicator cost as the base unit. Constellation parameters are derived from BUPT-1 [26] dataset. Given its 115 Wh battery and 25% Depth of Discharge (DoD) limit, we allocate $E^{proc}_{budget} = 1.5$ Wh and $E^{comm}_{budget} = 5.0$ Wh to ensure orbital sustainability. Hardware efficiency metrics align with the NVIDIA Jetson Orin Nano [26]. representing a modern accelerator suitable for LEO satellite deployment. Table 1 shows considered scenarios and constellation configurations. To ensure reproducibility, the complete framework implementation is made publicly available.[1].

The evaluation assesses Constella across four dimensions. First, the trade-off between system cost and inference success rate is examined to quantify the ability of Constella to identify minimal-cost configurations without sacrificing task completion. Second, end-to-end inference latency is compared across approaches to evaluate the effectiveness of adaptive routing over static assignment strategies. Third, total energy consumption per orbit is analyzed to assess the impact of constellation composition on the energy balance. Fourth, computational overhead of both offline and online phases is measured to confirm practical feasibility.

[1] https://github.com/stanisica/Constella.

4.2 Result analysis

The key question for an identified constellation configurations is whether cost is minimized without sacrificing inference task completion. Figure 2 shows that Constella maintains at least 81.9% success across all scenarios while reducing cost by one to two orders of magnitude compared to TB and NB. Success rate is defined as the fraction of generated images transmitted to the ground station within a single orbit. In extra-small and small scenarios, Constella achieves 100% success at minimal cost, while NB degrades in the small scenario (10% success) because its midpoint split layer produces an 8.96 Mbits intermediate tensor that overwhelms communicator buffers under static routing. In larger scenarios, Constella maintains high completion while requiring significantly lower cost. In the extra-large scenario, however, TB and NB degrade because their fixed processing and communication configurations exceed energy and buffer limits, whereas Constella adapts the split layer and constellation size to satisfy these constraints.

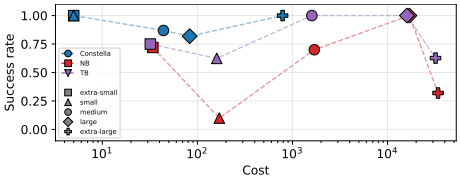

Fig. 2: Deployment cost vs. success rate.

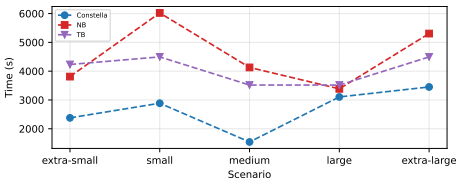

Fig. 3: Mean inference latency.

Beyond cost, timely delivery of inference results is essential for latency-sensitive applications. Figure 3 shows that Constella reduces mean latency by 1.1–2.7× compared to TB and NB across scenarios. Mean latency is computed as the average time from image capture to completion of transmission to the ground station, with missed tasks assigned a one-orbit penalty of $\mathcal{T} = 6300\ s$. In the medium scenario, Constella achieves the largest improvement, reaching 1541 s versus 4129 s for NB and 3518 s for TB. Its median latency of 991 s indicates that most tasks are delivered earlier, while a smaller tail waits for the next orbital pass. In the large scenario, latencies become more similar because Constella achieves its cost savings with only 3 communicators, which limits the routing advantage available over the more heavily provisioned baselines.

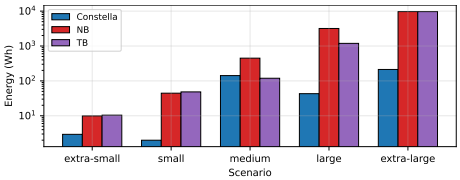

Fig. 4: Energy consumption per orbit.

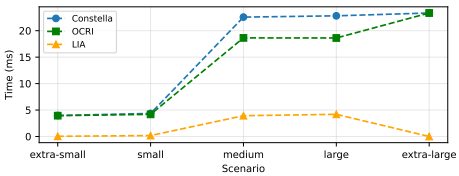

Fig. 5: Mean execution time per orbit.

Reducing the number of active satellites also has direct implications for energy consumption. Figure 4 shows that Constella achieves the lowest total energy consumption in all scenarios except medium, consuming 3.5–45.5× less energy than TB and up to 74× less than NB. These savings arise from jointly selecting an efficient split point and smaller constellation configurations, which reduces

both computation and communication energy. For example, in the large scenario, Constella uses only 23 satellites and consumes 43.0 Wh, compared to 3183.6 Wh for NB and 1191.0 Wh for TB. The only exception is the medium scenario, where TB consumes less total energy than Constella (119.1 Wh vs. 141.9 Wh) because it activates no communicators, while Constella expends additional communication energy to achieve substantially lower cost and latency.

Finally, we evaluate the execution overhead of OCRI and LIA to assess practical feasibility. Figure 5 shows mean execution times over 50 iterations. OCRI completes in 3.93–23.35 ms across all scenarios, demonstrating that the offline optimization remains efficient as problem complexity increases. The total LIA execution time shown in Figure 5 represents the cumulative time across all routing decisions per scenario, for one orbit, ranging from 0.04–4.19 ms. Dividing this range by the number of decisions per orbit yields a mean per-decision execution time of 0.50–4.04 μs, confirming negligible online overhead for real-time operation. In the extra-large scenario, OCRI selects a configuration with no communicators ($Y = 0$), so no LIA routing decisions are required and its execution time is zero. Across all scenarios, the combined Constella execution time remains under 28 ms, confirming practical feasibility.

5 Conclusion

Distributed inference in space data centers requires balancing deployment cost, energy consumption, and execution performance. This work introduced Constella, a two-phase framework for cost-efficient distributed inference in space. Constella jointly optimizes constellation configuration and split-layer selection offline, and performs adaptive processor-to-communicator assignment online. Evaluation shows that Constella maintains at least 81.9% inference success while achieving one to two orders of magnitude lower deployment cost, up to 2.7× lower mean latency, and up to 74× lower total energy consumption than TB and NB across scenarios. The execution overhead of OCRI and LIA remains low, with mean runtimes below 24 ms per configuration and 5 ms per orbit for the cumulative execution of all assignment decisions, respectively, confirming practical feasibility. Future work will extend the Constella by modeling inference time as a variable to account for hardware heterogeneity. Additionally, we will investigate the joint optimization of satellite role placement within the constellation topology to further improve system efficiency.

Acknowledgments.

This work was partly funded by the European Union under the Horizon Europe programme through the SNS JU (Grant Agreement No. 101192912, NexaSphere). Views expressed are those of the authors and do not necessarily reflect those of the EU or the SNS JU.

References

1. Adeel, A., Gogate, M., Farooq, S., Ieracitano, C., Dashtipour, K., Larijani, H., Hussain, A.: A survey on the role of wireless sensor networks and iot in disaster management. In: Geological disaster monitoring based on sensor networks, pp. 57–66. Springer (2018)

2. Agnew, M., Renouard, L., Hegyi, A.: Edrs-spacedatahighway: Near-real-time data relay services for leo satellites and haps. In: 30th AIAA International Communications Satellite System Conference (ICSSC). p. 15214 (2012)
3. Beals, T., Biggs, M., Bloom, J.V., Fischbacher, T., Gromov, K., Köster, U., Pravahan, R., Manyika, J., et al.: Towards a future space-based, highly scalable ai infrastructure system design. arXiv preprint arXiv:2511.19468 (2025)
4. Bhattacherjee, D., Singla, A.: Network topology design at 27,000 km/hour. In: Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies. pp. 341–354 (2019)
5. Caon, M., Ros, P.M., Martina, M., Bianchi, T., Magli, E., Membibre, F., Ramos, A., Latorre, A., Kerr, M., Wiehle, S., Breit, H., Günzel, D., Mandapati, S., Balss, U., Tings, B.: Very low latency architecture for earth observation satellite onboard data handling, compression, and encryption. In: 2021 IEEE International Geoscience and Remote Sensing Symposium IGARSS. pp. 7791–7794 (2021). https://doi.org/10.1109/IGARSS47720.2021.9554085
6. Chen, Y., Zhang, Q., Xing, R., Li, Y., Ma, X., Zhang, Y., Zhou, A., Wang, S.: Slice: Energy-efficient satellite-ground co-inference via layer-wise scheduling optimization. IEEE Transactions on Services Computing **18**(4), 2388–2402 (2025). https://doi.org/10.1109/TSC.2025.3577451
7. Denby, B., Lucia, B.: Orbital edge computing: Nanosatellite constellations as a new class of computer system. In: Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 939–954 (2020)
8. for Disaster Risk Reduction, U.N.O.: Global Assessment Report on Disaster Risk Reduction 2025: Resilience Pays: Financing and Investing for Our Future. Stylus Publishing, LLC (2025)
9. Dlamini, T., Vilakati, S.: Remote and rural connectivity: Infrastructure and resource sharing principles. Wireless Communications and Mobile Computing **2021**(1), 6065119 (2021)
10. Fossa, C.E., Raines, R.A., Gunsch, G.H., Temple, M.A.: A performance analysis of the iridium® low earth orbit satellite system with a degraded satellite constellation. SIGMOBILE Mob. Comput. Commun. Rev. **2**(4), 54–61 (Oct 1998). https://doi.org/10.1145/1321400.1321404
11. Heller, B., Sherwood, R., McKeown, N.: The controller placement problem. In: Proceedings of the First Workshop on Hot Topics in Software Defined Networks. pp. 7–12. HotSDN '12 (Aug 2012). https://doi.org/10.1145/2342441.2342444
12. Jia, X., Lv, T., He, F., Huang, H.: Collaborative data downloading by using inter-satellite links in LEO satellite networks. IEEE Transactions on Wireless Communications **16**(3), 1523–1534 (2017). https://doi.org/10.1109/TWC.2017.2647805
13. Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. Commun. ACM **60**(6), 84–90 (May 2017). https://doi.org/10.1145/3065386, https://doi.org/10.1145/3065386
14. Li, J., et al.: Collaborative last-hop scheduling strategy for large-scale LEO constellation routing. Computer Networks (2024). https://doi.org/10.1016/j.comnet.2024.110655
15. Monte, P.A., Louie, M., Wiedeman, R.: Mobile telephony through leo satellites: To obp or not. In: NASA. Lewis Research Center, Space Communications Technology Conference: Onboard Processing and Switching (1991)
16. Popa, L., Ratnasamy, S., Iannaccone, G., Krishnamurthy, A., Stoica, I.: A cost comparison of datacenter network architectures. In: Proceedings of the

6th International COnference. Co-NEXT '10, Association for Computing Machinery, New York, NY, USA (2010). https://doi.org/10.1145/1921168.1921189, https://doi.org/10.1145/1921168.1921189
17. Pusztai, T., Hisberger, J., Marcelino, C., Nastic, S.: Stardust: A scalable and extensible simulator for the 3d continuum. In: 2025 IEEE International Conference on Edge Computing and Communications (EDGE). pp. 44–53. IEEE (2025)
18. Pusztai, T., Marcelino, C., Nastic, S.: Hyperdrive: Scheduling serverless functions in the edge-cloud-space 3d continuum. In: 2024 IEEE/ACM Symposium on Edge Computing (SEC). pp. 265–278. IEEE (2024)
19. Santos, H.G., Toffolo, T.A.M.: Mixed Integer Linear Programming with Python (2020), https://python-mip.readthedocs.io/, `python-mip` user manual, version 1.6.2
20. Tan, M., Le, Q.: Efficientnet: Rethinking model scaling for convolutional neural networks. In: International conference on machine learning. pp. 6105–6114. PMLR (2019)
21. Wang, H., Ma, S., Dong, L., Huang, S., Zhang, D., Wei, F.: Deepnet: Scaling transformers to 1,000 layers. IEEE Transactions on Pattern Analysis and Machine Intelligence **46**(10), 6761–6774 (2024)
22. Wang, R., Kishk, M.A., Yang, H.H., Alouini, M.S.: Satellite-terrestrial routing or intersatellite routing? a stochastic geometry perspective. IEEE Transactions on Aerospace and Electronic Systems **61**(4), 8770–8786 (2025). https://doi.org/10.1109/TAES.2025.3544589
23. Wang, S., Zhang, M., Huang, X., Hu, T., Sun, Q.C., Corcoran, J., Liu, Y.: Urban–rural disparity of social vulnerability to natural hazards in australia. Scientific reports **12**(1), 13665 (2022)
24. Westphal, C., Han, L., Li, R.: LEO satellite networking relaunched: Survey and current research challenges. ITU Journal on Future and Evolving Technologies **4**, 711–744 (Dec 2023). https://doi.org/10.52953/LWXC1928
25. Wu, C., Han, S., Chen, Q., Wang, Y., Meng, W., Benslimane, A.: Enhancing leo mega-constellations with inter-satellite links: Vision and challenges. IEEE Wireless Communications (2025)
26. Xing, R., Xu, M., Zhou, A., Li, Q., Zhang, Y., Qian, F., Wang, S.: Deciphering the enigma of satellite computing with cots devices: Measurement and analysis. In: Proceedings of the 30th Annual International Conference on Mobile Computing and Networking. p. 420–435. ACM MobiCom '24 (2024). https://doi.org/10.1145/3636534.3649371
27. Xu, Z., Zhang, P., Li, C., Zhu, H., Xu, G., Sun, C.: A Collaborative Inference Algorithm in Low-Earth-Orbit Satellite Network for Unmanned Aerial Vehicle. Drones **7**(9), 575 (Sep 2023). https://doi.org/10.3390/drones7090575
28. Yao, S., Lin, Y., Wang, M., Xu, K., Xu, M., Xu, C., Zhang, H.: LEOEdge: A Satellite-Ground Cooperation Platform for the AI Inference in Large LEO Constellation. IEEE Journal on Selected Areas in Communications **43**(1), 36–50 (Jan 2025). https://doi.org/10.1109/JSAC.2024.3460083
29. Yep, T.: torchinfo (2020), https://github.com/TylerYep/torchinfo, view model summaries in PyTorch!
30. Zhang, Y., Lin, Z., Chen, Z., Fang, Z., Chen, X., Zhu, W., Zhao, J., Gao, Y.: Satfed: A resource-efficient leo-satellite-assisted heterogeneous federated learning framework. Engineering **54**, 115–126 (2025). https://doi.org/https://doi.org/10.1016/j.eng.2025.07.020
31. Zschau, J., Küppers, A.N.: Early warning systems for natural disaster reduction. Springer Science & Business Media (2013)